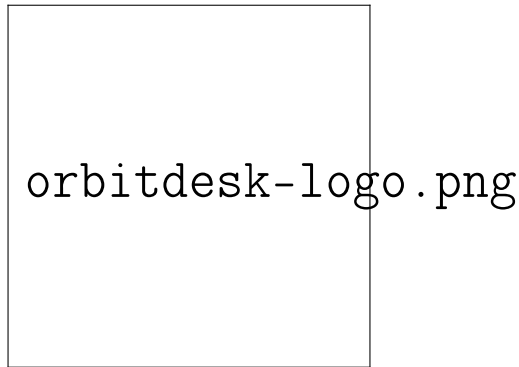




OrbitDesk

Complete System Requirements & Architectural Specification

NGO Project Management & Financial Governance Platform

Document Version: 2.0 (Complete Enterprise Edition)

Target Platform: NGO Project Management & Financial Governance Platform

Status: Full Reference Specification

August 8, 2026

This document serves as the complete architectural reference for the OrbitDesk enterprise platform, providing detailed specifications for all functional modules, system architecture, database schema, and non-functional requirements.

Table of Contents

1	System Overview	7
1.1	Introduction	7
1.2	Design Philosophy	7
1.3	Target Audience	7
1.4	Scope and Coverage	8
2	Hierarchical Structure	8
2.1	The Five-Tier Organization Model	8
2.2	Tier 1: Organization	8
2.2.1	Key Characteristics	8
2.2.2	Governance	8
2.3	Tier 2: Workspace	9
2.3.1	Key Characteristics	9
2.4	Tier 3: Project	9
2.4.1	Key Characteristics	9
2.5	Tier 4: Task	9
2.5.1	Key Characteristics	9
2.6	Tier 5: Subtask	10
2.7	Hierarchy Visualization	10
3	Unified Roles & Access Control (RBAC)	10
3.1	The Seven Core System Roles	10
3.1.1	Role 1: Owner	10
3.1.2	Role 2: Admin	11
3.1.3	Role 3: Coordinator	11
3.1.4	Role 4: Manager	12
3.1.5	Role 5: Finance Officer	12
3.1.6	Role 6: Member	13
3.1.7	Role 7: Viewer	13
3.2	Scope Rule of Precedence	14
3.3	Role Management Workflows	14
3.3.1	Invitation Flow	14
3.3.2	Token Expiration and Security	15
3.4	Permission Inheritance	15
4	Detailed Functional Modules	15
4.1	Organization Governance & Multi-Tenancy	15

4.1.1	Organization Profile Management	15
4.1.2	Soft Deletion Architecture	16
4.1.3	Self-Service Tokenized Invites	16
4.1.4	Ownership Transfer Protocol	16
4.1.5	Consortium & Partner Linking	17
4.1.6	Compliance Profile & Automated Reminders	17
4.2	Workspace & Reusable Team Operations	17
4.2.1	Workspace Management	17
4.2.2	Bulk Multi-Select Search Picker	18
4.2.3	Reusable Workspace Teams	18
4.2.4	Mid-Project Team Replacement Tracking	18
4.3	User Security, Authentication & Session Control	19
4.3.1	Multi-Factor Authentication (MFA)	19
4.3.2	Single Sign-On Integration	19
4.3.3	JWT Token Revocation Blacklist	19
4.3.4	Session & Device Management	20
4.3.5	Live Role Switcher Simulation	20
4.4	Project Lifecycle, Leadership Continuity & Planning	20
4.4.1	Project Lead Tenure Tracking	20
4.4.2	Project Status Lifecycle	21
4.4.3	Auditable Timeline Postponements	21
4.4.4	Logframe / Results Framework	21
4.4.5	Risk & Issue Register	22
4.5	Task Execution, Workflow & Google Calendar Integration	23
4.5.1	Kanban Workflow	23
4.5.2	Attributed Task Status History	23
4.5.3	Deadline vs. Completion Date	24
4.5.4	In-App Document Viewer	24
4.5.5	Cloud Storage Integration	24
4.5.6	Google Calendar Integration	25
4.5.7	Gantt / Timeline View	26
4.6	Financial Governance, Double-Entry Ledger & Bank Accounts	26
4.6.1	Full Double-Entry General Ledger	26
4.6.2	Dedicated Bank Account Management	27
4.6.3	USAID Allowable Expense Categorization	27
4.6.4	Multi-Currency Engine & Live Exchange Rates	28
4.6.5	Multi-Level Budgeting	28
4.6.6	Two-Step Expense Sign-off	29

4.6.7	Donor CRM & Contribution Tracking	29
4.6.8	1-Click Audit Support Package Export	30
4.7	Volunteers & External Portals	30
4.7.1	Volunteer Registry	30
4.7.2	In-Kind Hours Logging	31
4.7.3	Public Volunteer Portal	31
4.7.4	Public Contact Inquiry & Support Desk	31
4.8	Communications, Comments & Automation	32
4.8.1	Project vs. Task Comment Separation	32
4.8.2	Automated Background Worker Suite	32
4.9	Search, Analytics & Caching	33
4.9.1	Analytics & Dashboards	33
4.9.2	Global Faceted Search	33
4.9.3	Redis Caching Architecture	34
5	Database Schema Specification	34
5.1	Identity & RBAC Domain (13 Entities)	35
5.1.1	Core Entities:	35
5.1.2	Invitation and Membership:	35
5.1.3	Security and Governance:	35
5.2	Workspaces, Projects & Tasks Domain (15 Entities)	35
5.2.1	Workspace and Team Management:	35
5.2.2	Project Management:	36
5.2.3	Task Management:	36
5.2.4	Communications and Attachments:	36
5.3	Financial Management & Ledger Domain (7 Entities)	36
5.4	Grants & Donor CRM Domain (6 Entities)	37
5.5	Volunteers & M&E Logframe Domain (9 Entities)	37
5.5.1	Volunteer Management:	37
5.5.2	Risk and Issues:	37
5.5.3	Logframe (Results Framework):	37
5.6	Security Audit, Support & Search Domain (4 Entities)	37
6	Non-Functional & Security Architecture	38
6.1	Authentication & Authorization	38
6.1.1	Password Security	38
6.1.2	JWT Implementation	38
6.1.3	Field-Level Access Control	38
6.1.4	API Authorization	38

6.2	Data Governance & Immutability	39
6.2.1	Runtime Immutability Enforcement	39
6.2.2	Immutable Entities	39
6.3	Data Protection	39
6.3.1	Encryption at Rest	39
6.3.2	Encryption in Transit	39
6.3.3	Backup and Recovery	40
6.4	Localization Framework	40
6.4.1	Language Support	40
6.4.2	Localization Coverage	40
6.5	High Availability & Scalability	40
6.5.1	Horizontal Scaling	40
7	Technology Stack	41
7.1	Frontend Technologies	41
7.2	Backend Technologies	41
7.3	Database	41
7.4	DevOps & Infrastructure	41
8	Deployment Architecture	42
8.1	Production Environment	42
8.2	Staging Environment	42
8.3	Development Environment	42
9	API Strategy	42
9.1	API Design Principles	43
9.2	Authentication & Authorization	43
9.3	API Documentation	43
10	Data Migration & Seeding	43
10.1	Initial Data Seeding	43
10.2	Migration Strategy	44
10.3	Bulk Data Import	44
A	Appendix A: Glossary of Terms	45
B	Appendix B: Abbreviations	45
C	Appendix C: Revision History	46
D	Appendix D: References	46

List of Tables

1 System Overview

1.1 Introduction

OrbitDesk is an enterprise web-based project management and financial governance platform purpose-built for non-governmental organizations (NGOs), civil society organizations (CSOs), and international development agencies. The platform unifies standard program workflows—Kanban boards, Gantt timelines, task hierarchies, and activity reporting—with NGO-specific financial accountability including double-entry general ledger accounting, donor tracking, multi-level budgeting, USAID compliance categorization, grant reporting, and volunteer management.

1.2 Design Philosophy

The system is architected around three core principles:

1. **Hierarchical Clarity** — Every piece of work and financial data exists within a clearly defined organizational tier, ensuring that reporting, permissions, and oversight scale logically from the macro to the micro level.
2. **Financial Integrity** — All financial transactions are tracked through a double-entry general ledger system with immutable audit trails, ensuring complete accountability for donor funds and organizational expenditures.
3. **User-Centric Workflows** — The platform adapts to the way NGOs work, providing flexible tools for program staff, finance teams, volunteers, and leadership without forcing unnatural processes.

1.3 Target Audience

The OrbitDesk platform serves the following organizational types and stakeholders:

- **International NGOs** — Managing multiple country offices and donor portfolios
- **National NGOs** — Coordinating multi-department programs
- **Civil Society Organizations** — Requiring grant compliance and reporting
- **Development Agencies** — Overseeing consortium partnerships
- **Community-Based Organizations** — Seeking structured project governance

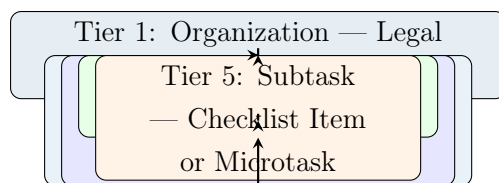
1.4 Scope and Coverage

This document covers **100% of Base Features** plus **12 Extended Enterprise Modules** and **Google Calendar Integration**. All specifications provided herein are complete and ready for implementation.

2 Hierarchical Structure

2.1 The Five-Tier Organization Model

OrbitDesk enforces a strict five-tier hierarchical structure that mirrors the operational reality of most NGOs. Every user, project, task, and financial record is anchored to a specific position within this hierarchy, enabling proper permissions, reporting, and oversight.



2.2 Tier 1: Organization

The Organization represents the legal NGO entity, country office, or headquarters. It is the topmost container within the system and serves as the boundary for multi-tenancy.

2.2.1 Key Characteristics

- Legal name, registration number, and tax-exemption certificates
- Base currency and operational timezone
- Branding and logo assets
- Compliance documentation and renewal schedules
- Contact information and headquarters address

2.2.2 Governance

An Organization can have exactly one **Owner** and multiple **Admins**, **Coordinators**, **Finance Officers**, and other roles assigned at the organization scope.

2.3 Tier 2: Workspace

Workspaces represent operational units within an Organization—such as departments, programme areas, field offices, or regional branches.

2.3.1 Key Characteristics

- Departmental budget ceilings and spending authority
- Reusable teams that can be assigned to multiple projects
- Workspace-level risk registers and compliance monitoring
- Visibility settings (Public, Private, Restricted)

2.4 Tier 3: Project

Projects are funded initiatives with defined start and end dates, budget ceilings, and donor linkages. Projects live within Workspaces and serve as the primary unit for program management.

2.4.1 Key Characteristics

- Detailed project charter and scope documentation
- Logframe (Results Framework) with goals, outcomes, outputs, and activities
- Budget allocations and donor contribution mappings
- Project-specific teams and leadership assignments

2.5 Tier 4: Task

Tasks are units of deliverable work assigned to team members, staff, or volunteers.

2.5.1 Key Characteristics

- Description, priority, and estimated effort
- Planned deadline and actual completion date
- Kanban status and status history
- Attachments, comments, and Google Calendar synchronization

2.6 Tier 5: Subtask

Subtasks represent checklist items or microtasks that make up a larger Task. They provide granular tracking for complex deliverables and can be assigned to specific individuals. Subtasks inherit the permissions and visibility of their parent Task.

2.7 Hierarchy Visualization

The system visualizes this hierarchy through breadcrumb navigation in the UI, allowing users to understand their current context within the organizational structure. All API endpoints enforce scope-level authorization checks to ensure users cannot access data outside their assigned permissions.

3 Unified Roles & Access Control (RBAC)

3.1 The Seven Core System Roles

OrbitDesk employs a comprehensive Role-Based Access Control (RBAC) system with seven distinct roles. A user holds a role at a specific scope (Organization, Workspace, or Project), and their permissions derive from the role-scope combination. This allows fine-grained access control while maintaining operational flexibility.

3.1.1 Role 1: Owner

Scope Authority: Organization

The **Owner** possesses ultimate authority over the entire Organization. This role has the highest privilege level and is reserved for the executive director, board chair, or country director depending on the organizational structure.

Key Responsibilities and Capabilities:

- Transfer ownership to another user through a formal, audited workflow
- Permanently delete the Organization and all associated data
- Override any financial sign-off, including both approval steps
- Set organization-wide policies including MFA enforcement and compliance standards
- Appoint and remove Administrators

Constraints: There is exactly one **Owner** per Organization, and this role cannot be assigned through standard invitation—it must be transferred through the formal ownership transfer protocol.

3.1.2 Role 2: Admin

Scope Authority: Organization

Admins handle the organizational setup and configuration. They serve as the operational backbone of the system, managing user lifecycle, workspace creation, and policy enforcement.

Key Responsibilities and Capabilities:

- Manage the member directory and user profiles
- Create, modify, and archive Workspaces
- Assign and revoke roles for users across the Organization
- Configure compliance profiles and document retention policies
- Set up MFA and SSO policies for the Organization
- Link partner NGOs (consortium relationships)
- View all users and system activity at the Organization level

Key Restriction: **Admins** cannot view, set, or reset user passwords. All authentication is handled through secure token-based invitations and password recovery flows.

3.1.3 Role 3: Coordinator

Scope Authority: Workspace

Coordinators manage the operational flow across multiple Projects within a Workspace. This role is typically held by programme managers, department heads, or field office directors.

Key Responsibilities and Capabilities:

- Coordinate Projects across a Workspace
- Manage reusable Teams at the Workspace level
- Assign volunteers to Projects and Tasks
- Oversee the Workspace Risk & Issue Register

- Manage cross-project scheduling and resource allocation
- Approve time-off requests and capacity adjustments

3.1.4 Role 4: Manager

Scope Authority: Project

Managers run individual Projects day-to-day. This is the most active role in the system, responsible for delivering programmatic results.

Key Responsibilities and Capabilities:

- Create Projects, Tasks, and Subtasks
- Manage Kanban boards and workflow states
- Assign task deadlines and priorities
- Draft project budgets and revise as needed
- Perform second-step expense sign-offs (after Finance Officer approval)
- Add and remove team members from the Project
- Generate and distribute project reports

3.1.5 Role 5: Finance Officer

Scope Authority: Organization / Workspace / Project

Finance Officers own the financial governance of the system. They ensure that all financial activities are properly recorded, categorized, and compliant with donor requirements.

Key Responsibilities and Capabilities:

- Maintain the double-entry general ledger
- Manage bank accounts and cash positions
- Perform first-step expense approvals
- Allocate donor contributions to Projects and Tasks
- Ensure USAID compliance categorization
- Generate audit exports and financial reports
- Set financial category rules and thresholds

Scope Application: A **Finance Officer** can operate at Organization level (overseeing all financial activity), Workspace level (managing departmental budgets), or Project level (focused on specific initiative finances).

3.1.6 Role 6: Member

Scope Authority: Project

Members perform assigned work within Projects. This role represents the majority of active system users—program staff, technical experts, and field personnel.

Key Responsibilities and Capabilities:

- Update Task and Subtask statuses
- Post comments and @mention other users
- Attach files and link Google Drive/OneDrive documents
- Log volunteer hours and work time
- View dashboards and reports for their projects
- Read access to financial information (view-only)

Key Restriction: **Members** have read-only access to management work created by others—they cannot create, approve, or delete organizational records outside their assigned tasks.

3.1.7 Role 7: Viewer

Scope Authority: Organization / Workspace / Project

Viewers provide read-only stakeholder access to specific scopes. This role is ideal for donors, board members, auditors, and partner organizations who need visibility but not editing privileges.

Key Responsibilities and Capabilities:

- View all system data within granted scopes
- Access reports, dashboards, and analytics
- Download documents and exports
- Monitor project progress and financial health

Key Restriction: **Viewers** cannot create, edit, approve, or delete any system records.

3.2 Scope Rule of Precedence

OrbitDesk implements a hierarchical scope resolution that ensures users have the right permissions in the right context.

The Rule: If a user holds conflicting roles at different scopes (for example, **Viewer** at Organization level but **Manager** on Project A), the more specific (lower-level) scope wins for that Project. The Organization-level role remains the permission floor everywhere else.

Example Application:

- User holds **Viewer** role at Organization level
- User holds **Manager** role on Project A
- User holds **Coordinator** role on Project B

Result:

- For Project A: User has full **Manager** permissions
- For Project B: User has **Coordinator** permissions
- For all other Projects: User has **Viewer** permissions (Organization-level floor)
- For Workspace-level views: User has **Viewer** permissions

This approach enables flexible role assignments without requiring complex role recalculations for every operation.

3.3 Role Management Workflows

3.3.1 Invitation Flow

Admins and **Owners** invite users via email with expiring tokens. The process follows these steps:

1. Admin enters the user's email address and selects the desired role-scope combination
2. System generates a secure, time-limited invitation token
3. An automated email is sent to the user with a registration link
4. User clicks the link and sets their own password (Admin never sees or sets passwords)
5. Upon successful registration, the user is assigned the specified role-scope combination
6. All actions are logged to the immutable Audit Log

3.3.2 Token Expiration and Security

- Invitation tokens expire after 7 days by default (configurable at Organization level)
- Failed login attempts trigger progressive delays
- Password reset tokens expire after 1 hour
- JWT tokens can be revoked via the token blacklist

3.4 Permission Inheritance

Permissions flow downward through the hierarchy but can be overridden at lower levels:

- **Organization-level permissions** apply to all Workspaces and Projects unless explicitly overridden
- **Workspace-level permissions** apply to all Projects within that Workspace unless overridden
- **Project-level permissions** apply only to that specific Project

When a user is assigned a role at a higher level (e.g., Organization Admin), they can operate at that level across the entire system. When assigned at a lower level, their permissions are restricted to that specific scope.

4 Detailed Functional Modules

4.1 Organization Governance & Multi-Tenancy

4.1.1 Organization Profile Management

The Organization profile serves as the central configuration point for the NGO entity. Administrators and Owners can manage:

- Legal name and registration number
- Tax-exemption certificates and compliance documents
- Logo and branding assets
- Base currency (default: USD)
- Timezone and operational country
- Description and mission statement

- Contact information and headquarters address

4.1.2 Soft Deletion Architecture

OrbitDesk implements a soft-deletion pattern for all major entities. When an Organization is “deleted,” it is not removed from the database but marked as deleted with a timestamp. The complete state is preserved as a JSON backup ([BackupJson](#)) for audit and recovery purposes.

Benefits:

- Data can be restored in case of accidental deletion
- Historical relationships remain intact for reporting
- Audit logs maintain complete context
- Regulatory compliance is maintained

4.1.3 Self-Service Tokenized Invites

The invitation system is fully self-service and tokenized:

- Admins and Owners generate invites through the UI
- Each invite includes a unique, cryptographically secure token
- Tokens expire after a configurable period
- Users set their own passwords upon first login
- All invitations are logged with status tracking

4.1.4 Ownership Transfer Protocol

Formal ownership transfer follows a secure workflow:

1. Current Owner initiates a transfer request
2. System generates a confirmation token
3. New Owner receives the token via email (or in-person secure delivery)
4. New Owner confirms the transfer with the token
5. System logs the transfer to the immutable Audit Log
6. Role assignments are updated atomically

4.1.5 Consortium & Partner Linking

OrbitDesk supports multi-entity implementation through [OrganizationPartner](#) linking:

- Link partner NGO organizations
- Define shared project implementation arrangements
- Maintain partner-specific reporting requirements
- Enable cross-organization data sharing with controlled access

4.1.6 Compliance Profile & Automated Reminders

The Compliance Profile stores all regulatory and operational documentation:

- Tax-exemption certificates and renewal dates
- Registration documents from various jurisdictions
- USAID compliance certifications
- Audit letters and financial statements
- Grant-specific compliance requirements

Automated Reminders: The `ComplianceReminderService` runs as a background worker, sending alerts for:

- Document expiration (30, 14, 7, and 1 days before)
- Renewal dates for certifications
- Required reporting deadlines

4.2 Workspace & Reusable Team Operations

4.2.1 Workspace Management

Workspaces serve as containers for related Projects, Teams, and resources. Full CRUD operations include:

- **Create:** Define a new Workspace with name, description, and visibility level
- **Read:** View Workspace details, member lists, and associated Projects
- **Update:** Modify Workspace properties, budget ceilings, and visibility settings
- **Archive:** Soft-archive Workspaces that are no longer active
- **Delete:** Purge Workspace data (restricted to Owners)

Visibility Levels:

- **Public:** Visible to all Organization members
- **Private:** Visible only to Workspace members
- **Restricted:** Visible only to explicitly invited users

4.2.2 Bulk Multi-Select Search Picker

The `MultiSelectMembers` component provides powerful user selection:

- Type-ahead search across the organization directory by name or email
- Multi-select capability for adding multiple users at once
- Selection of teams, roles, and individual users
- Used for adding members to Workspaces, Teams, and Projects
- Optimized for large organization directories (up to 10,000+ users)

4.2.3 Reusable Workspace Teams

Teams are named groups of users defined at the Workspace level:

- Designate Team Leads with management authority
- Reusable across multiple Projects within the Workspace
- Simplify member assignment by adding entire Teams to Projects
- Maintain historical membership records

Benefits:

- Reduce administrative overhead
- Ensure consistent team composition across Projects
- Enable rapid project setup
- Maintain continuity when staff members change

4.2.4 Mid-Project Team Replacement Tracking

When teams or members need to be replaced during a project, OrbitDesk maintains complete historical records:

- `ProjectTeamHistory` tracks all team assignments
- `RemovedAt` timestamp when a team is replaced

- `ReplacedByTeamID` links to the replacement team
- Full audit trail of all changes

Use Case: When a implementing partner leaves a project mid-cycle, the replacement team is logged with the reason for change, ensuring continuity and accountability.

4.3 User Security, Authentication & Session Control

4.3.1 Multi-Factor Authentication (MFA)

MFA is supported at three enforcement levels:

1. **Enforced:** All users must enable MFA before accessing the system
2. **Optional:** Users can enable MFA for enhanced security
3. **Disabled:** MFA is not available (not recommended)

Supported Methods:

- Time-based One-Time Passwords (TOTP) via authenticator apps
- Backup codes for recovery scenarios
- Email-based OTP (fallback)

4.3.2 Single Sign-On Integration

OrbitDesk integrates with Google Workspace SSO:

- Standard OAuth 2.0 flow
- `GoogleSignIn.jsx` frontend component
- `GoogleCallback.jsx` for authorization code exchange
- Automatic user provisioning from Google directory (optional)
- Seamless authentication experience

4.3.3 JWT Token Revocation Blacklist

The system maintains a database-backed revocation blacklist:

- `RevokedToken` entity stores JTI tokens
- Revocation timestamps for audit trail
- Immediate invalidation of logged-out sessions

- Support for forced logout of all sessions

4.3.4 Session & Device Management

Users can view and manage their active sessions:

- List of all active sessions with device information
- Location and timestamp of each session
- Revoke individual sessions remotely
- Force logout of compromised devices

4.3.5 Live Role Switcher Simulation

The frontend `RoleSwitcherBar.jsx` component provides:

- Real-time UI preview as any of the 7 system roles
- For Admins and Managers only (testing and auditing)
- Dynamic permission rendering
- No actual permission changes—just UI simulation
- Safe for training and demonstration

4.4 Project Lifecycle, Leadership Continuity & Planning

4.4.1 Project Lead Tenure Tracking

Project Lead assignments are recorded as historical records:

- `ProjectLeadHistory` entity stores all lead assignments
- Each assignment has `StartDate` and `EndDate`
- Reassigning the lead automatically closes the old record (`EndDate` = current timestamp)
- New record created with the new lead's information
- Complete leadership history queryable

Benefits:

- Clear accountability timeline
- Support for leadership transitions
- Audit trail for donor reporting

- Historical reporting accuracy

4.4.2 Project Status Lifecycle

Projects follow a defined status lifecycle:

1. **Planning:** Initial development phase, budgets being drafted, team forming
2. **Active:** Full implementation underway
3. **OnHold:** Temporarily paused (funding gap, staffing issues, etc.)
4. **Completed:** Work finished, reporting in progress
5. **Cancelled:** Project terminated before completion
6. **Archived:** Completed projects moved to archival storage

Status Transitions:

- Status changes are logged with timestamp and user
- Certain transitions require specific role permissions
- [ProjectPostponement](#) records track timeline delays

4.4.3 Auditable Timeline Postponements

When project timelines shift, the system requires formal logging:

- **Old Date:** Original scheduled date
- **New Date:** Revised scheduled date
- **Reason:** Justification for the postponement
- **Requester:** User initiating the postponement
- **Approver:** Authorizing user (usually Program Director or Country Director)

All postponements are logged and visible in audit reports.

4.4.4 Logframe / Results Framework

OrbitDesk implements a complete M&E (Monitoring and Evaluation) structure:

Hierarchy:

- **Goal:** Highest-level change objective (e.g., “Improved food security”)
- **Outcome:** Medium-term results (e.g., “Increased household income”)

- **Output:** Direct deliverables (e.g., “60 farmers trained in sustainable agriculture”)
- **Activity:** Specific actions (e.g., “Conduct 5 training workshops”)

KPIs and Indicators:

- Each level can have multiple KPIs
- **Baseline:** Starting value
- **Target:** Desired achievement
- **Actual:** Measured progress
- Automatic progress aggregation from linked tasks

UI Components:

- `LogframeController` handles backend logic
- `LogframeView.jsx` provides visual representation
- Hierarchical display with expandable sections

4.4.5 Risk & Issue Register

Comprehensive risk and issue management with quantitative scoring:

Risk Matrix:

- **Likelihood:** Score 1–5 (Rare to Almost Certain)
- **Impact:** Score 1–5 (Insignificant to Catastrophic)
- **Risk Score:** Likelihood $\times$ Impact
- **Mitigation Plans:** Detailed strategies and owners
- **Status:** Open, Mitigating, Resolved, Accepted

Issue Log:

- **Description:** Problem statement
- **Severity:** Critical, Major, Minor, Trivial
- **Root Cause:** Analysis of underlying cause
- **Resolution:** Steps taken to resolve
- **Owner:** Responsible person

Organization-Level Risk Rollup: The `OrgRiskRollup.jsx` dashboard aggregates risks from all Workspaces and Projects, providing leadership with a complete view of organizational risk exposure.

4.5 Task Execution, Workflow & Google Calendar Integration

4.5.1 Kanban Workflow

The interactive Kanban board provides visual workflow management:

Standard Columns:

1. **ToDo:** Pending tasks, not yet started
2. **InProgress:** Active work in progress
3. **InReview:** Completed work awaiting review
4. **Blocked:** Tasks blocked by external factors
5. **Done:** Completed work

Features:

- Drag-and-drop task movement
- Real-time updates across users
- WIP (Work In Progress) limits per column
- Custom column configuration (advanced)

4.5.2 Attributed Task Status History

Complete state transition logging:

- **Old Status:** Previous workflow state
- **New Status:** Updated workflow state
- **Changed By:** User ID making the change
- **Timestamp:** Exact time of change
- **Change Reason:** Optional justification

Immutability: Status history records are strictly immutable—they cannot be modified or deleted after creation.

4.5.3 Deadline vs. Completion Date

OrbitDesk separates planned and actual dates:

- **Deadline:** Planned completion date (user-set)
- **CompletedDate:** Actual completion timestamp (system-set when task moves to Done)
- Variance reporting (late/early completion)
- Burn-rate and schedule performance analysis

4.5.4 In-App Document Viewer

The `InAppFileViewer.jsx` component renders documents directly in the browser:

Supported Formats:

- Images: JPEG, PNG, GIF, WebP, SVG
- Video: MP4, WebM, OGG
- Audio: MP3, WAV, OGG
- PDF: Full document rendering
- Office: DOCX, XLSX, PPTX (read-only preview)
- Text: Plain text, CSV, JSON, XML

Features:

- No mandatory file downloads
- Responsive display for all screen sizes
- Zoom and pan for detailed viewing
- Password-protected documents (optional)

4.5.5 Cloud Storage Integration

Seamless integration with cloud storage providers:

Google Drive:

- Link files directly from Google Drive
- Maintain version history
- Collaborative editing support

Microsoft OneDrive:

- Attach OneDrive files
- Permission-aware access
- Enterprise integration

AttachmentList.jsx Component:

- Unified view of all attachments
- File type icons and preview
- Download and share options

4.5.6 Google Calendar Integration

Three-layer integration for calendar synchronization:

1. Direct Web URL Generation:

- 1-click link to `calendar.google.com/calendar/render`
- Pre-populated with task details
- No API required, works for all users

2. iCal File Download:

- GET `/api/v1/tasks/{id}/ical` endpoint
- Standard .ics format
- Compatible with all calendar applications
- Recurring task support

3. Direct API Sync:

- POST `/api/v1/tasks/{id}/sync-google-calendar`
- Pushes events directly to Google Calendar
- Authentication via OAuth 2.0
- Updates and deletions synchronized
- Bidirectional sync support

4.5.7 Gantt / Timeline View

The Gantt chart provides project scheduling visualization:

Features:

- Task durations with start/end dates
- Dependencies (Finish-to-Start, Start-to-Start)
- Critical path highlighting
- Resource allocation display
- Postponement overlays
- Milestone markers

Dependencies:

- [TaskDependency](#) entity tracks relationships
- Constraint-based scheduling
- Automatic recalculation on changes
- Drag-to-create dependencies

4.6 Financial Governance, Double-Entry Ledger & Bank Accounts

4.6.1 Full Double-Entry General Ledger

The [FinancialTransaction](#) entity implements complete double-entry accounting:

Transaction Components:

- **Nominal Amount:** Amount in transaction currency
- **Base Currency Amount:** Amount in organization's base currency
- **Exchange Rate:** Rate used for conversion
- **Transaction Type:** Expense, Income, Transfer, Adjustment
- **Payee/Payer Details:** Counterparty information
- **Reference Number:** Internal or external reference
- **Description:** Transaction purpose
- **Date:** Transaction date

Accounting Structure:

- Debit and Credit entries for each transaction
- Account codes and chart of accounts
- Trial balance calculation
- General ledger reporting

4.6.2 Dedicated Bank Account Management

The [BankAccount](#) entity tracks all financial accounts:

Account Details:

- Account name and number
- SWIFT/BIC code
- IBAN (international)
- Current balance (with optimistic concurrency)
- Currency denomination

Operations:

- Bank-to-bank cash transfers ([ToBankAccountId](#))
- Direct donor deposit mappings
- Reconciliation with transactions
- Account closing and archiving

Concurrency Control:

- [RowVersion](#) field for optimistic locking
- Prevent double-spending scenarios
- Transactional integrity guarantees

4.6.3 USAID Allowable Expense Categorization

The [FinancialCategory](#) entity implements USAID compliance:

Chart of Accounts Levels:

1. **Asset:** Resources owned by the organization

2. **Liability:** Obligations to other entities
3. **Equity:** Net assets and reserves
4. **Revenue:** Income from donors and other sources
5. **Expense:** Costs incurred in operations

Compliance Features:

- **IsUSAIDAllowable:** Flag for grant regulations
- **Category Type:** Direct cost, Indirect cost, Administrative
- **Threshold Rules:** RequiresReceiptThreshold (e.g., \$500)
- **Subcategory:** Detailed categorization within each level

4.6.4 Multi-Currency Engine & Live Exchange Rates

The `CurrencyController` manages all currency operations:

- Dynamic rate conversion between currencies
- Donor pledge currency to organization base currency
- Organization base currency to local disbursement currencies
- Exchange rate management with historical tracking
- Automatic rate updates (via API or manual)

4.6.5 Multi-Level Budgeting

Budgets exist at multiple organizational levels:

Budget Levels:

- **Organization:** Overall budget ceiling
- **Workspace:** Departmental allocations
- **Project:** Individual project budgets
- **Task:** Detailed activity budgets

Budget Management:

- `BudgetLineItem`: Individual budget entries
- **Overspend Validation:** Prevention of exceeding limits

- **Revision History:** [BudgetRevisionLog](#) tracks all changes
- **Roll-up Reporting:** Consolidation across levels

4.6.6 Two-Step Expense Sign-off

Expense approval requires two independent reviews:

Step 1: Finance Officer Review

- Initial approval and validation
- Ensure proper documentation
- Verify category and accounting treatment
- Confirm availability of funds

Step 2: Manager Sign-off

- Programmatic review
- Ensure expense is project-appropriate
- Second approval for donor reporting

Override: [Owner](#) can override either step in exceptional circumstances (logged to audit trail).

4.6.7 Donor CRM & Contribution Tracking

Complete donor management and tracking:

Donor Profiles:

- Name, type (Government, Foundation, Corporate, Individual)
- Contact information
- Compliance requirements
- Communication history

Contribution Management:

- Pledged vs received tracking
- Contribution schedule and terms
- Project and task allocations

- Matching fund tracking

Communication Log:

- [DonorCommunication](#) entity
- Meeting notes and correspondence
- Compliance reporting updates

4.6.8 1-Click Audit Support Package Export

The `ComplianceController` provides:

- Full financial trail extraction
- ZIP archive generation
- All supporting documentation
- External auditor format
- Export tracking and logging

4.7 Volunteers & External Portals

4.7.1 Volunteer Registry

Comprehensive volunteer management:

Volunteer Profiles:

- Contact information
- Skills inventory and certifications
- Availability schedule
- Language proficiency
- Emergency contact information

Task Assignment:

- [TaskVolunteer](#) association
- Hours tracking and logging
- Performance feedback

4.7.2 In-Kind Hours Logging

`VolunteerHour` entity tracks all contributed time:

- Date and duration (hours/minutes)
- Task and project association
- Description of work performed
- Value calculation for donor reporting
- In-kind contribution reporting

4.7.3 Public Volunteer Portal

Self-service volunteer application:

- `/volunteer-apply`: Public-facing application form
- `VolunteerPublicApply.jsx`: React component
- **Features:**
 - Profile creation without staff user account
 - Skills and interest declaration
 - Availability scheduling
 - Automated follow-up emails
 - Internal review workflow

4.7.4 Public Contact Inquiry & Support Desk

External contact handling:

`ContactInquiry`:

- Name and email of inquirer
- Subject and message
- Category (General inquiry, Partnership, Complaint, etc.)
- Priority assignment

Support Ticketing:

- `IsResolved` flag
- Internal notes for staff

- Resolution tracking
- Reply functionality via `ContactController`
- Email reply integration

4.8 Communications, Comments & Automation

4.8.1 Project vs. Task Comment Separation

Comments carry `EntityType` and `EntityId` to maintain context:

- **Project Comments:** Governance discussions, decisions, updates
- **Task Comments:** Execution notes, clarifications, blockers

Threading:

- `ParentCommentID` for replies
- Threaded display in UI
- Conversation tracking

@Mentions:

- User notification on mention
- Auto-complete for user names
- Unread mention tracking

Edit History:

- Comment modification tracking
- Original version preservation

4.8.2 Automated Background Worker Suite

Three background workers handle routine automation:

1. `ComplianceReminderService`:

- Scans for upcoming document expirations
- Sends automated email alerts
- Escalates to Admin if not resolved

2. RecurringTaskWorkerService:

- Generates recurring tasks based on templates
- Maintains task continuity
- Supports daily, weekly, monthly, quarterly schedules

3. ScheduledReportWorkerService:

- Generates reports on defined schedules
- Sends PDF/Excel exports via email
- Report distribution lists

4.9 Search, Analytics & Caching

4.9.1 Analytics & Dashboards

Comprehensive dashboard suite:

Task Analytics:

- Task completion rate
- Average cycle time
- Burndown charts
- Workload distribution

Budget vs Spend Analytics:

- Budget utilization percentages
- Forecasting and projections
- Variance analysis

Workload Distribution:

- `WorkloadChart.jsx` component
- Individual vs team capacity
- Resource allocation

4.9.2 Global Faceted Search

Scoped search across all entities:

Searchable Entities:

- Projects (name, description, status)
- Tasks (title, description, assignee)
- Donors (name, type, status)
- Financial Records (transaction, budget)
- Volunteers (name, skills)
- Documents (content, metadata)

Saved Searches:

- [SavedSearch](#) entity
- Reuse common search patterns
- Share with teams

4.9.3 Redis Caching Architecture

High-performance caching layer:

Cached Items:

- User permissions and roles
- Navigation structures
- Session metadata
- Frequently accessed entities

Benefits:

- Reduced database load
- Faster page loads
- Scalability for large organizations

5 Database Schema Specification

OrbitDesk utilizes a normalized SQL Server relational schema containing 54 entities organized into functional domains.

5.1 Identity & RBAC Domain (13 Entities)

The identity domain manages users, authentication, authorization, and organizational membership. This domain forms the foundation for all access control decisions.

5.1.1 Core Entities:

- **Organization** — The top-level legal entity container, including all metadata and configuration
- **User** — Individual system users with authentication credentials and profile information
- **Role** — Defined system roles (Owner, Admin, Coordinator, Manager, Finance Officer, Member, Viewer)
- **AppPermission** — Granular system permissions associated with specific actions
- **RoleAssignment** — Assignment of a Role to a User at a specific Scope
- **RolePermission** — Link between Role and AppPermission (what actions each role can perform)

5.1.2 Invitation and Membership:

- **OrganizationMember** — User membership in an Organization
- **OrganizationInvitation** — Pending invitations to join the Organization
- **UserInvitation** — Generalized invitation tracking

5.1.3 Security and Governance:

- **RevokedToken** — JWT token revocation blacklist
- **OwnershipTransferRequest** — Formal ownership transfer workflow tracking
- **OrganizationPartner** — Linked partner NGO relationships
- **OrganizationCompliance** — Compliance documentation and renewal tracking

5.2 Workspaces, Projects & Tasks Domain (15 Entities)

The operational domain manages all programmatic work and organizational structure.

5.2.1 Workspace and Team Management:

- **Workspace** — Department, programme area, or field office container
- **Team** — Reusable named teams at Workspace level

- [TeamMember](#) — Individual membership in a Team

5.2.2 Project Management:

- [Project](#) — Funded initiative with defined scope and timeline
- [ProjectLeadHistory](#) — Historical records of project leadership tenure
- [ProjectTeamHistory](#) — Team assignment history with replacement tracking
- [ProjectPostponement](#) — Formal timeline postponement records
- [ProjectTeam](#) — Current team assignments for a Project

5.2.3 Task Management:

- [TaskItem](#) — Individual task within a Project
- [TaskStatusHistory](#) — Immutable task status transition records
- [Subtask](#) — Checklist items under a Task
- [TaskMember](#) — User assignments to specific Tasks
- [TaskDependency](#) — Relationships between Tasks (Finish-to-Start, Start-to-Start)

5.2.4 Communications and Attachments:

- [Comment](#) — User comments with threading, @mentions, and edit history
- [Attachment](#) — File attachments with cloud storage links

5.3 Financial Management & Ledger Domain (7 Entities)

The financial domain implements double-entry accounting and budget management.

- [BankAccount](#) — Financial institution accounts with SWIFT/IBAN tracking
- [FinancialCategory](#) — Chart of Accounts with USAID compliance flags
- [FinancialTransaction](#) — Double-entry general ledger transactions
- [Budget](#) — Multi-level budget containers
- [BudgetLineItem](#) — Individual budget entries within a Budget
- [BudgetRevisionLog](#) — Immutable audit trail of budget changes
- [Expense](#) — Expense records with approval workflow tracking

5.4 Grants & Donor CRM Domain (6 Entities)

The grants and donor domain manages funding relationships and compliance.

- **Donor** — Funding organization or individual profiles
- **ProjectDonor** — Donor association to specific Projects
- **DonorContribution** — Pledged and received contributions
- **DonorCommunication** — History of donor communications
- **GrantCondition** — Specific conditions and requirements for grants
- **GrantReportSchedule** — Scheduled reporting requirements for grants

5.5 Volunteers & M&E Logframe Domain (9 Entities)

The monitoring and volunteer management domain tracks both human resources and program results.

5.5.1 Volunteer Management:

- **Volunteer** — Volunteer profiles and skills registry
- **TaskVolunteer** — Volunteer assignments to specific Tasks
- **VolunteerHour** — Hours tracking for in-kind reporting

5.5.2 Risk and Issues:

- **RiskIssue** — Risk Matrix and Issue log entries with scoring

5.5.3 Logframe (Results Framework):

- **LogframeGoal** — Highest-level program goals
- **LogframeOutcome** — Intermediate results
- **LogframeOutput** — Direct deliverables
- **LogframeActivity** — Specific activities
- **Indicator** — KPIs with baseline, target, and actual values

5.6 Security Audit, Support & Search Domain (4 Entities)

- **AuditLog** — Immutable system activity log
- **Notification** — User notifications and alerts

- [SavedSearch](#) — User-saved search queries
- [ContactInquiry](#) — External contact form submissions

6 Non-Functional & Security Architecture

6.1 Authentication & Authorization

6.1.1 Password Security

- bcrypt password hashing with work factor 12
- Salt included in hash for rainbow table protection
- Password complexity requirements (minimum 8 characters, mixed case, numbers, special characters)
- Password history prevents reuse of last 5 passwords

6.1.2 JWT Implementation

- Bearer token authentication
- Short-lived access tokens (15 minutes)
- Refresh token rotation
- Token revocation blacklist ([RevokedToken](#) entity)
- JTI (JWT ID) for token tracking

6.1.3 Field-Level Access Control

- Granular permissions at field level
- Read/write based on user role and scope
- Sensitive field filtering (e.g., financial details)

6.1.4 API Authorization

- RBAC boundary checks at every endpoint
- Scope verification for each operation
- Pre-flight permission checks

- Consistent error handling

6.2 Data Governance & Immutability

6.2.1 Runtime Immutability Enforcement

- `OrbitDbContext.cs SaveChanges` interceptor
- Prevents updates on critical audit entities
- Prevents deletions of historical records
- Maintains audit trail integrity

6.2.2 Immutable Entities

The following entities are strictly immutable:

- `AuditLog`
- `TaskStatusHistory`
- `ProjectLeadHistory`
- `ProjectTeamHistory`
- `ProjectPostponement`
- `FinancialTransaction` (append-only, no updates)

6.3 Data Protection

6.3.1 Encryption at Rest

- Database-level encryption (TDE)
- Encrypted sensitive fields
- Secure key management

6.3.2 Encryption in Transit

- TLS 1.3 for all communications
- HSTS enforcement
- Perfect forward secrecy

6.3.3 Backup and Recovery

- Automated daily backups
- Point-in-time recovery
- Geo-redundant backup storage
- Backup integrity verification

6.4 Localization Framework

6.4.1 Language Support

- Full English and Amharic language support
- Extensible architecture for additional languages
- `TranslationController` for API-based translations
- `TranslationContext.jsx` for React component translations
- `LanguageSwitcher.jsx` for user preference management

6.4.2 Localization Coverage

- UI labels and buttons
- System messages and notifications
- Email templates
- Report generation
- Error messages

6.5 High Availability & Scalability

6.5.1 Horizontal Scaling

- Stateless application tier
- Load balancer distribution
- Read-replica database optimization
- Auto-scaling for demand fluctuations

7 Technology Stack

7.1 Frontend Technologies

- **Framework:** React.js with TypeScript
- **State Management:** Redux Toolkit
- **UI Library:** Material-UI (MUI)
- **Internationalization:** i18next
- **Charts:** Recharts, Chart.js
- **Calendar:** FullCalendar
- **Gantt:** DHTMLX Gantt
- **File Viewer:** react-pdf, officegen

7.2 Backend Technologies

- **Framework:** ASP.NET Core 8.0
- **Language:** C# 12.0
- **API:** RESTful with Swagger/OpenAPI
- **Authentication:** JWT, OAuth 2.0
- **Caching:** Redis
- **Background Jobs:** Hangfire
- **Email:** SendGrid / SMTP

7.3 Database

- **Main Database:** Microsoft SQL Server
- **ORM:** Entity Framework Core
- **Migrations:** EF Core Migrations

7.4 DevOps & Infrastructure

- **CI/CD:** GitHub Actions / Azure DevOps
- **Containerization:** Docker
- **Orchestration:** Kubernetes (optional)

- **Monitoring:** Application Insights
- **Logging:** Serilog
- **Cloud:** Microsoft Azure / AWS

8 Deployment Architecture

8.1 Production Environment

- **Web Server:** IIS / Kestrel with reverse proxy (Nginx)
- **Load Balancer:** Azure Load Balancer / AWS ELB
- **Application Tier:** 3+ instances for high availability
- **Database Tier:** Primary with read replicas
- **Cache Tier:** Redis cluster
- **Storage:** Azure Blob Storage / AWS S3

8.2 Staging Environment

- Mirror of production with reduced scale
- Integration testing environment
- UAT environment
- Performance testing environment

8.3 Development Environment

- Local development with Docker Compose
- Shared development database
- Feature branch deployments

9 API Strategy

9.1 API Design Principles

- RESTful design with resource-oriented URLs
- Consistent naming conventions
- Versioning through URL path (/api/v1/)
- Standard HTTP status codes
- Comprehensive error responses
- Pagination for all list endpoints
- Filtering and sorting support

9.2 Authentication & Authorization

- Bearer token authentication
- Token-based role evaluation
- Scope verification per request

9.3 API Documentation

- Swagger/OpenAPI 3.0
- Interactive API explorer
- Example requests and responses
- Authentication testing

10 Data Migration & Seeding

10.1 Initial Data Seeding

- System roles and permissions
- Default financial categories
- Sample projects and tasks (optional)
- Template configurations

10.2 Migration Strategy

- Entity Framework Core migrations
- Automatic migration execution on deployment
- Rollback capability
- Data transformation scripts for version upgrades

10.3 Bulk Data Import

- CSV/Excel import for organizations
- Batch import for users
- API-based import for integrations

A Appendix A: Glossary of Terms

- **NGO:** Non-Governmental Organization
- **CSO:** Civil Society Organization
- **RBAC:** Role-Based Access Control
- **MFA:** Multi-Factor Authentication
- **SSO:** Single Sign-On
- **JWT:** JSON Web Token
- **USAID:** United States Agency for International Development
- **GL:** General Ledger
- **KPI:** Key Performance Indicator
- **M&E:** Monitoring and Evaluation
- **SWIFT:** Society for Worldwide Interbank Financial Telecommunication
- **IBAN:** International Bank Account Number

B Appendix B: Abbreviations

- **API:** Application Programming Interface
- **CRM:** Customer Relationship Management
- **CRUD:** Create, Read, Update, Delete
- **ORM:** Object-Relational Mapping
- **TOTP:** Time-based One-Time Password
- **UAT:** User Acceptance Testing
- **WIP:** Work In Progress

C Appendix C: Revision History

-
- **Version 1.0:** Initial draft — Core requirements
 - **Version 1.5:** Extended modules added
 - **Version 2.0:** Complete enterprise edition with all features

D Appendix D: References

- ISO 9001:2015 Quality Management Systems
- USAID Automated Directives System (ADS) Chapters
- Generally Accepted Accounting Principles (GAAP)
- OAuth 2.0 Authorization Framework (RFC 6749)
- OpenID Connect Core 1.0
- GDPR Compliance Requirements

OrbitDesk — Complete System Requirements & Architectural Specification

Document Version 2.0 (Complete Enterprise Edition)

© 2024 OrbitDesk. All Rights Reserved.